Naia Conference Game - Specification

A timed challenge where hydropower operators spot production losses on a Naia-style dashboard by drawing expected production lines, classifying each loss by category and subcategory, and estimating loss size. Built as a standalone local-first app with the same look & feel as the real Naia platform.

Spec Documents

1. Game Concept & Rules - Core gameplay (draw losses, classify, estimate), 3-level scoring, 5-minute standard timer
2. Architecture & Tech Stack - Local-first standalone app, SQLite + Google Sheets sync, Infomaniak SMTP email, French UI
3. Scenario Preparation - How Bernard creates scenarios (CSV upload, loss marking, intro videos, maintenance events)
4. Game Flow & UX - Screen-by-screen walkthrough (START → video → GO → play → summary → email → results → classement), on-screen keyboard, signup page
5. Registration, Scoreboard & Email - Post-game registration, Classement (leaderboard), results email with CTA, Naia signup page
6. Technical Implementation - Data model, drawing layer, scoring algorithm, API, frontend components
7. Conference Setup - Hardware (single PC, no keyboard), local startup, staffing, fallback plans
8. Hardware Build: Tabletop Arcade - Poplar plywood 15mm cabinet, lightbox logo panel, panel dimensions, transport crate

Key Decisions

- **Standalone local-first app** — runs on the game PC without internet. The developer stays focused on the product. Google Sheets sync for cloud backup when WiFi available.
- **Draw-to-identify game** — players draw expected production lines on the chart to identify losses, then classify by category and subcategory. Loss size is scored by accuracy of the drawn area.
- **Gameplay screen = Naia dashboard** — the gameplay screen is 100% identical to the real Naia platform, doubling as a live product demo. Only a chronometer is added.
- **Register after play** — no friction before the game. Email capture happens after gameplay when the player wants detailed results.
- **3-level scoring** — category (+10/-5), subcategory (+5/-2), size estimation (0-10 based on accuracy 80-100%)
- **Single mode: 5 minutes** — one standard time for all players, fair leaderboard
- **French UI** — all screens in French including the Classement (leaderboard)
- **No keyboard** — all text input via on-screen virtual keyboard (AZERTY)
- **Infomaniak SMTP** — email results via naia.energy domain, no extra API service needed
- **CTA to naia.energy/signup** — results screen and email include a call-to-action linking to a signup page (part of this project)
- **Poplar plywood 15mm** — ~50% lighter than MDF 18mm, same structural strength
- **Illuminated logo panel** — lightbox with opal acrylic + LED strip for eye-catching branding
- **Multi-console support** — 2 arcade cabinets share a combined leaderboard via Google Sheets with write verification. Each console works offline, syncs when WiFi is available
- **Scoreboard on same PC** — second browser window on a connected TV via 5m HDMI cable
- **Built with Claude Code** — issues and test cases defined upfront, then worked through one by one

---

01 - Game Concept & Rules

Elevator Pitch

"Can you spot and diagnose what's costing this plant money?" — A timed challenge where hydropower operators look at a Naia-style dashboard showing production, rain, and river flow, draw expected production lines to identify losses, classify each loss by category and subcategory, and estimate loss size. The fastest and most accurate diagnostician wins.

Goals

1. **Engagement**: Draw conference attendees to the Naia booth with a competitive, fun challenge
2. **Discovery**: Let operators experience Naia's core value proposition (loss identification = lost revenue) hands-on
3. **Lead generation**: Capture contact details of qualified prospects (plant operators)
4. **Brand recall**: Participants walk away remembering Naia as "the tool that shows you where you're losing money"

Core Gameplay

Game Flow

1. **START** — Player presses the start button on the welcome screen
2. **Intro Video** — Short video showing the plant (configurable per scenario, stored locally). Gives the player context: location, river, capacity, typical operation
3. **GO!** — Player presses "GO" when ready
4. **Gameplay** — Draw loss areas, classify causes, estimate sizes (5 minutes)
5. **Summary Result** — Quick score overview with rank
6. **Add Email** — On-screen keyboard to enter email address for detailed results
7. **Classement** — Live leaderboard + button to start a new game. Detailed results sent by email only

What the Player Sees

A Naia-style dashboard showing a plant over ~2 months:

- **Production bars** (daily energy, kWh) — the main chart
- **Rain indicator** (inverted bars above the chart) — precipitation pattern
- **River flow indicator** (line overlaid) — water availability
- **Maintenance event dots** — clickable dots on the production line showing interventions (e.g., "restart turbine", "clean grille"). These give contextual clues
- **Left panel** with toggleable additional indicators (extra flow stations, comparators)

What the Player Does

1. **Identify losses**: Draw a line on the production graph indicating what production *should* have been. The gap between this line and actual production = the loss area
2. **Classify each loss**: Select the **category** (6 options) and **subcategory** (3-5 options per category) from a dropdown
3. **The loss size is automatically calculated** from the drawn line vs. actual production (surface area between the curves)
4. Repeat for all losses they can find within the time limit

What the Player Is Reasoning About

They see...	Clues to use	Likely cause
High rain + high flow + zero production	Flow was there, production stopped suddenly	Breakdown
Zero production on a calm, normal day	No weather event, perhaps a maintenance dot nearby	Maintenance
Extreme rain + extreme flow + zero production	Flood conditions visible in rain/flow data	Environmental
Production at 50% despite full flow	No external reason, operator choice	Operational
Flat-capped production despite high flow	Comparator plant also capped → grid issue	Grid/Constraint
Zero readings but flow is normal, gap pattern	Maintenance dot says "sensor replaced"	Data Issue

Maintenance Event Dots

Clickable dots placed on the production line at specific dates. When clicked, a popup shows a short maintenance note (e.g., "Restart turbine", "Clean intake grille", "Replace flow sensor"). These:

- Give contextual clues for diagnosis (a "restart turbine" dot next to a zero-production period → breakdown)
- Mirror real Naia where operators log interventions
- Are provided by Bernard in the scenario data

Scoring System

Scoring is on **3 levels** per identified loss area:

Level 1: Category (max 10 pts per loss)

Diagnosis	Points
Correct category	+10
Wrong category	-5

Level 2: Subcategory (max 5 pts per loss)

Diagnosis	Points
Correct subcategory	+5
Wrong subcategory	-2

Level 3: Loss Size Estimation (max 10 pts per loss)

Based on accuracy of the drawn line vs. Bernard's reference loss size:

Accuracy	Points	Formula
100%	10	`max(0, (accuracy - 0.80) / 0.20 × 10)`
98%	9	Rounded to nearest integer
90%	5	
80%	0	
<80%	0	

Accuracy = `1 - abs(player_loss_size - actual_loss_size) / actual_loss_size`

The "loss size" is the area (in kWh) between the player's drawn expected-production line and the actual production bars, for the days covered by that loss.

Maximum per Loss Area

A perfectly identified loss scores: 10 (category) + 5 (subcategory) + 10 (size) = **25 points**. Plus time bonus/penalty on top.

Penalties for False Losses

If a player marks an area as a loss where Bernard has not defined one: **-5 points** (to discourage guessing).

Time Bonus / Penalty

Each scenario has a **normal completion time** parameter (configured by Bernard, e.g., 180 seconds). This is the expected time for a competent operator to finish.

Situation	Points
Faster than normal	**+1 point per second** faster
Slower than normal	**-1 point per 5 seconds** slower

Examples (normal = 180s):

- Finished in 150s → 30 seconds faster → **+30 points**
- Finished in 200s → 20 seconds slower → **-4 points** (20/5 = 4)

- Finished in 300s → 120 seconds slower → **-24 points**

The time limit still applies (game ends when timer runs out), but the time bonus/penalty is based on completion time vs. normal completion time, not vs. the time limit.

Final Score

```
score = sum(per_loss_points) + time_bonus_or_penalty
```

Loss Categories & Subcategories

#	Category	Subcategories
1	**Breakdown**	Turbine failure · Generator failure · Electrical fault · Intake/canal blockage · Other mechanical
2	**Maintenance**	Planned inspection · Overhaul/repair · Cleaning · Regulatory inspection
3	**Environmental**	Flood · Ice/frost · Debris · Drought/low water · Sediment
4	**Operational**	Reservoir management · Testing/commissioning · Manual stop · Reduced operation
5	**Grid/Constraint**	Grid outage · Curtailment · Regulatory limit · Voltage/frequency issue
6	**Data Issue**	Meter gap · Calibration error · Communication loss · Sensor failure

These map to the real Naia loss taxonomy (7 categories with 69 subtypes) but are simplified for a timed game.

Timing

Mode	Time Limit	Expected Losses	Notes
Standard	5 min	5-8 losses	Single mode. All players get the same time

Fixed at 5 minutes. No mode switching — keeps it simple and fair for the leaderboard.

Why This Works for Operators

- **Familiar context**: They read these exact patterns every day on their own plants
- **Competitive**: "I know more about hydro than you" — operators are proud of their expertise
- **Low barrier**: The intro video gives enough context even for non-experts
- **Three skill layers**: Finding the loss, classifying the cause, and estimating the size — rewards deep expertise
- **Aha moment**: Even if they get some wrong, they realize "a tool that does this automatically would save me time and money"

---

02 - Architecture & Tech Stack

Decision: Local-First Standalone App

The game is built as a **standalone application** that runs **locally** on the game PC at the conference. It replicates the Naia dashboard look & feel but has its own codebase, its own data, and its own deployment.

Why Local-First

Factor	Benefit
Conference WiFi is unreliable	Game works without internet. No loading delays, no outages. Players never wait.

Developer focus	The developer stays 100% on the product (issues 215-225, security). The game can be built by Claude Code or a separate person.
Zero production risk	Game bugs cannot affect real customers. No shared database, no shared deployment.
Simple deployment	Copy folder to game PC, run `npm start`. No cloud infrastructure to manage on-site.
Instant response	Local SQLite = sub-millisecond queries. No network latency on the chart or scoring.

Data Strategy: PostgreSQL vs. SQLite

The main Naia platform uses PostgreSQL + TimescaleDB. For the conference game, two options were considered:

Factor	PostgreSQL (reuse Naia stack)	SQLite + Google Sheets
Offline support	❌ Requires running PG server	✅ File-based, always works
Setup complexity	❌ PG install + config on game PC	✅ Zero-config, npm install
Code reuse	⚠️ Some Drizzle schemas reusable	❌ Different schema, fresh code
Deployment	❌ PG service + Nuxt server	✅ Single `node` process
Backup	❌ pg_dump needed	✅ Copy one file
Cloud sync	Built-in (if online)	Via Google Sheets API
Data volume	Overkill (<1000 rows)	Perfect fit

Decision: SQLite + Google Sheets. The game has minimal data (participants + scores), needs to run fully offline, and must be deployable by copying a folder. PostgreSQL adds complexity without benefit here. Google Sheets provides cloud backup and real-time remote monitoring when WiFi is available.

```
Game PC (primary)                    Cloud (backup + remote access)
┌──────────────────┐                 ┌──────────────────────┐
│ SQLite (game.db) │ ─── sync ──→    │ Google Sheets        │
│ All game data    │                  │ Participants + scores│
│ Scenarios (JSON) │                  │ Real-time view       │
│ Videos (local)   │                  │ CRM-ready export     │
└──────────────────┘                 └──────────────────────┘
```

- **SQLite**: Primary data store. All game state, participants, scores
- **Google Sheets**: Background sync after each game session. Serves as cloud backup and allows remote monitoring (e.g., from a phone)
- **Sync is fire-and-forget**: If internet is down, data is safe in SQLite. Sync catches up when connection returns

Multi-Console Support (2 Arcade Cabinets)

Both consoles run independently with their own SQLite database. Google Sheets acts as the **shared backend** for the combined leaderboard.

```
Console 1                        Console 2
┌──────────────────┐             ┌──────────────────┐
│ Nuxt server      │             │ Nuxt server      │
│ SQLite (local)   │             │ SQLite (local)   │
│ Game + classement│             │ Game + classement│
└──────────┬───────┘             └──────────┬───────┘
           │ write + verify                 │ write + verify
           ▼                                ▼
      ┌──────────────────────────────────────────┐
```

```
    │         Google Sheets (cloud)                │
    │  Shared leaderboard — both consoles          │
    │  read from here for the combined classement  │
    └──────────────────────────────────────────────┘
         │                                  │
         ▼                                  ▼
    TV (classement)                    TV (classement)
    reads from GSheets                 reads from GSheets
    via its console                    via its console
```

Why no LAN? The game PCs (laptops) may not have ethernet ports, and conference WiFi is the only network available. Google Sheets over WiFi is simpler than setting up a local network.

Sync reliability: Since conference WiFi can be unreliable, the sync includes **write verification** — after writing a row, the console reads it back to confirm. If verification fails, the row is re-queued. See chapter 05 for details.

Fallback: If WiFi is completely down, each console shows its own local leaderboard (SQLite only). When WiFi returns, both sync and the combined leaderboard rebuilds automatically.

Classement: Second Browser Window on Same PC

No separate computer needed for the scoreboard. Each game PC runs two browser windows:

1. **Game window** (primary screen) — the gameplay interface
2. **Classement window** (secondary screen/TV) — `/classement` page in a separate Chrome window

Both served by the same local Nuxt server on `localhost:3000`.

What We Reuse from Naia

- **Visual design**: Same color scheme, fonts, layout patterns, Nuxt UI components
- **Chart library**: uPlot for rendering production bars, rain, and flow indicators
- **Domain concepts**: Loss categories, indicator overlays, aggregated daily view
- **Gameplay screen**: 100% identical to the real Naia dashboard — the game doubles as a live product demo. Only a chronometer is added on top.

What We Don't Reuse

- Backend (FastAPI, Celery, Alembic) — replaced by lightweight Node/Nuxt server
- Database (PostgreSQL + TimescaleDB) — replaced by SQLite
- Authentication (JWT, RBAC) — replaced by simple game session tokens
- Data ingestion pipeline — scenarios are pre-processed, loaded as static JSON

Tech Stack

Frontend

- **Nuxt 3** (Vue 3 + TypeScript) — same as Naia, consistent look
- **Nuxt UI** — same component library as Naia
- **uPlot** — same chart library as Naia, extended with drawing interaction
- **Pinia** — state management for game state

Backend

- **Nuxt server routes** (`server/api/`) — no separate backend needed
- Nuxt 3's built-in server (Nitro) handles API routes, keeping everything in one project

Database

- **SQLite** (via `better-sqlite3` or Drizzle ORM) — lightweight, zero-config, file-based
- Perfect for a conference game: <1000 records total, no concurrent write pressure

- Easy to backup (copy one file), easy to reset

Cloud Sync

- **Google Sheets API** — append rows after each game session
- Uses a service account key (JSON file on the game PC)
- Sheets document shared with the team for real-time monitoring

Video

- **Local MP4/WebM files** — stored in `scenarios/SC-XXX/intro.mp4`
- HTML5 `<video>` element, no streaming service needed

Email

- **Infomaniak SMTP** — send results email via authenticated SMTP using the naia.energy domain
- Server: `mail.infomaniak.com`, port 587 (STARTTLS)
- Uses Nodemailer with SMTP transport (standard Node.js library)
- Credentials stored in `.env` on the game PC
- Sent from: `game@naia.energy`
- Queued locally if offline, sent when connection returns

> ***Note**: Infomaniak (our hosting provider for naia.energy) does not offer a transactional email API like Gmail API or Resend. Instead, it provides standard SMTP access on `mail.infomaniak.com:587` with STARTTLS authentication. This is simpler than an API approach — just configure Nodemailer with SMTP credentials. No OAuth, no API keys, no Google Cloud Console setup needed.*

UI Language

All screens are in French. The game targets French-speaking hydropower operators at conferences in France, Belgium, and Switzerland. All UI labels, buttons, messages, error texts, and the scoreboard ("Classement") are in French.

Project Structure

```
naia-conference-game/
├── app/                        # Nuxt 3 app directory
│   ├── pages/
│   │   ├── index.vue           # Welcome / attract screen ("Bienvenue")
│   │   ├── intro.vue           # Intro video player
│   │   ├── play.vue            # Main gameplay (drawing + diagnosis)
│   │   ├── results.vue         # Score breakdown + email input + CTA
│   │   ├── classement.vue      # Public leaderboard (2nd screen)
│   │   └── admin.vue           # Staff admin panel
│   ├── components/
│   │   ├── game/
│   │   │   ├── Chart.vue       # uPlot chart with drawing interaction
│   │   │   ├── DrawingLayer.vue  # Canvas overlay for line drawing
│   │   │   ├── IndicatorPanel.vue # Left panel: toggle additional indicators
│   │   │   ├── MaintenanceDot.vue # Clickable maintenance event popup
│   │   │   ├── CauseSelector.vue # Category + subcategory picker popup
│   │   │   ├── LossList.vue    # Bottom list of identified losses
│   │   │   ├── Timer.vue       # Countdown timer / chronometer
│   │   │   ├── ScoreBar.vue    # Top bar: timer + score + progress
│   │   │   ├── CountdownOverlay.vue # 3-2-1-GO
│   │   │   ├── ResultsBreakdown.vue # Post-game details per loss
│   │   │   ├── RevenueImpact.vue # EUR display
│   │   │   ├── OnScreenKeyboard.vue # Virtual keyboard for email input
│   │   │   └── CTAPanel.vue    # Sign up button + new game button
│   │   └── scoreboard/
```

```
│   │       ├── Leaderboard.vue   # Top 10 table ("Classement")
│   │       └── NowPlaying.vue    # Current game status
│   ├── composables/
│   │   ├── useGameState.ts       # Game flow state machine
│   │   ├── useGameChart.ts       # Chart data loading + indicator toggling
│   │   ├── useDrawing.ts         # Line drawing on chart + loss area calculation
│   │   ├── useScoring.ts         # Client-side score display
│   │   └── useTimer.ts           # Countdown logic
│   └── stores/
│       └── game.ts               # Pinia store
│
├── server/
│   ├── api/
│   │   ├── start.post.ts
│   │   ├── submit.post.ts
│   │   ├── results/[id].get.ts
│   │   ├── send-results.post.ts  # Send results email
│   │   ├── scoreboard.get.ts
│   │   └── admin/
│   │       ├── participants.get.ts
│   │       ├── export.get.ts
│   │       └── settings.post.ts
│   ├── db/
│   │   ├── schema.ts             # Drizzle ORM schema
│   │   ├── index.ts              # DB connection (SQLite)
│   │   └── seed.ts               # Initial data
│   ├── services/
│   │   ├── scoring.ts            # Scoring engine (server-side, authoritative)
│   │   ├── scenarios.ts          # Load scenario data from JSON files
│   │   ├── sheets-sync.ts        # Google Sheets background sync
│   │   └── email.ts              # Send results email via Infomaniak SMTP
│   └── utils/
│       └── loss-area.ts          # Calculate loss area from drawn line
│
├── scenarios/                    # Game scenario data
│   ├── SC-001/
│   │   ├── config.json
│   │   ├── production.json
│   │   ├── rain.json
│   │   ├── flow.json
│   │   ├── indicators.json       # Additional available indicators
│   │   ├── maintenance.json      # Maintenance event dots
│   │   ├── losses.json           # Answer key (server-only)
│   │   └── intro.mp4             # Plant intro video
│   ├── SC-002/
│   │   └── ...
│   └── ...
│
├── tools/
│   └── prepare-scenario.ts       # CLI: Bernard's CSV → JSON converter
│
├── public/
│   └── images/                   # Naia logo, category icons
│
├── nuxt.config.ts
├── package.json
```

```
├── .env.example
└── start.bat                    # Double-click to start on Windows
```

Deployment: Game Day

Primary: Run Locally

```
# On the game PC (Windows/Mac/Linux)
cd naia-conference-game
npm run build
node .output/server/index.mjs
# Game at http://localhost:3000
# Classement at http://localhost:3000/classement (open on 2nd screen)
```

Or with a convenience script (`start.bat` on Windows):

```
@echo off
cd /d "%~dp0"
node .output/server/index.mjs
```

Scoreboard on Second Screen

1. Connect a TV/monitor as extended display
2. Open Chrome on the second screen: `http://localhost:3000/classement`
3. Press F11 for fullscreen

No network, no separate computer, no HDMI stick needed.

Cloud Backup

Google Sheets syncs in the background. If WiFi is available, scores appear in the shared sheet within seconds. If not, they sync when connection returns.

---

03 - Scenario Preparation

This document describes how Bernard (or any domain expert) creates game scenarios.

Overview

A scenario = one plant, a couple of months of data, with loss areas defined (size, category, subcategory) and maintenance events marked. Bernard prepares this using real (anonymized) or realistic data.

What Bernard Provides

1. Plant Info

- Plant name (fictional, e.g., "Centrale de Moulins")
- Capacity in kW (e.g., 450)
- River name (e.g., "La Dordogne")
- Country

2. Intro Video

A short video (30-90 seconds) showing the plant: the river, the intake, the powerhouse, the turbine hall. This gives players context before they play.

- Format: MP4 or WebM
- Resolution: 1920x1080 recommended
- Stored locally in the scenario folder
- Can be filmed on a phone — doesn't need to be professional

3. Production Data (CSV)

Daily or sub-daily power/energy data for the scenario period (~60 days).

Expected CSV format:

```
datetime,value
2025-01-01 00:00,120.5
2025-01-01 00:15,118.3
2025-01-01 00:30,121.0
...
```

Or daily aggregated:

```
date,energy_kwh
2025-01-01,8540
2025-01-02,8230
2025-01-03,7980
...
```

The preparation tool will accept either format and aggregate to daily if needed.

4. Rain Data (CSV)

Daily rainfall from a nearby weather station.

```
date,rain_mm
2025-01-01,0.0
2025-01-02,12.3
2025-01-03,24.1
...
```

5. River Flow Data (CSV)

Daily average river flow from a hydrometric station.

```
date,flow_m3s
2025-01-01,8.2
2025-01-02,9.1
2025-01-03,14.5
...
```

6. Loss Areas (CSV) — Answer Key

This is the answer key. Bernard marks which days/periods are losses, what caused them, and the loss size.

```
start_date,end_date,category,subcategory,expected_kwh_per_day,description
2025-01-04,2025-01-04,breakdown,turbine_failure,8500,"Turbine trip — zero production despite normal flow"
2025-01-07,2025-01-08,environmental,flood,8500,"Flood event — intake submerged, safety shutdown"
2025-01-12,2025-01-12,maintenance,planned_inspection,8200,"Planned annual inspection"
```

```
2025-01-15,2025-01-16,grid_constraint,grid_outage,8300,"Grid outage in the area"
2025-01-19,2025-01-19,operational,reservoir_management,8400,"Reservoir flush ordered by authority"
2025-01-25,2025-01-25,data_issue,meter_gap,8100,"Flow sensor communication loss — data gap"
```

Fields:

- `start_date` , `end_date` : The period of the loss
- `category` : One of: `breakdown` , `maintenance` , `environmental` , `operational` , `grid_constraint` , `data_issue`
- `subcategory` : The specific subcategory (see table below)
- `expected_kwh_per_day` : What production *should* have been (Bernard's expert estimate). The tool uses this to calculate loss size
- `description` : Explanation shown to the player in results

Valid subcategories:

Category	Subcategories
breakdown	`turbine_failure`, `generator_failure`, `electrical_fault`, `intake_blockage`, `other_mechanical`
maintenance	`planned_inspection`, `overhaul_repair`, `cleaning`, `regulatory_inspection`
environmental	`flood`, `ice_frost`, `debris`, `drought_low_water`, `sediment`
operational	`reservoir_management`, `testing_commissioning`, `manual_stop`, `reduced_operation`
grid_constraint	`grid_outage`, `curtailment`, `regulatory_limit`, `voltage_frequency`
data_issue	`meter_gap`, `calibration_error`, `communication_loss`, `sensor_failure`

7. Maintenance Events (CSV)

Clickable informational dots on the production timeline. These give players contextual clues.

```
date,label
2025-01-05,Restart turbine after trip
2025-01-09,Clear debris from intake
2025-01-12,Start annual inspection
2025-01-13,End inspection — restart
2025-01-20,Normal operation resumed
2025-01-26,Replace flow sensor
```

Guidelines for maintenance events:

- Place them near loss areas to give clues (e.g., "restart turbine" after a breakdown)
- Include some events that are *not* near losses (normal operations) to avoid making it too easy
- Use short, operator-style language
- Aim for 8-15 events per scenario

Preparation Tool

A CLI tool converts Bernard's CSVs into game-ready JSON:

```
npx tsx tools/prepare-scenario.ts \
  --id SC-001 \
  --name "Centrale de Moulins" \
  --capacity 450 \
  --river "La Dordogne" \
  --country France \
  --production data/SC-001_production.csv \
```

```
  --rain data/SC-001_rain.csv \
  --flow data/SC-001_flow.csv \
  --losses data/SC-001_losses.csv \
  --maintenance data/SC-001_maintenance.csv \
  --video data/SC-001_intro.mp4 \
  --electricity-price 0.14
```

What the Tool Does

1. **Reads and validates** all CSV files
2. **Aggregates** sub-daily production to daily energy (kWh) if needed
3. **Aligns** all series to the same date range
4. **Calculates loss size** per loss area: `sum((expected_kwh_per_day - actual_kwh) for each day in range)`
5. **Calculates revenue loss**: `loss_size_kwh × electricity_price`
6. **Assigns loss zone numbers** (Loss 1, Loss 2, etc.) in chronological order
7. **Copies the intro video** to the scenario folder
8. **Outputs** scenario JSON files to `scenarios/SC-XXX/`

Output: Scenario JSON Files

`scenarios/SC-001/config.json`

```json
{
  "id": "SC-001",
  "plantName": "Centrale de Moulins",
  "capacityKw": 450,
  "river": "La Dordogne",
  "country": "France",
  "periodStart": "2025-01-01",
  "periodEnd": "2025-02-28",
  "difficulty": "standard",
  "timeLimitSec": 300,
  "normalCompletionTimeSec": 180,
  "numLosses": 6,
  "totalRevenueLossEur": 11630,
  "introVideo": "intro.mp4"
}
```

`scenarios/SC-001/production.json`

```json
[
  { "date": "2025-01-01", "energyKwh": 8540 },
  { "date": "2025-01-02", "energyKwh": 8230 }
]
```

`scenarios/SC-001/rain.json`

```json
[
  { "date": "2025-01-01", "rainMm": 0.0 },
  { "date": "2025-01-02", "rainMm": 12.3 }
]
```

`scenarios/SC-001/flow.json`

```
[
  { "date": "2025-01-01", "flowM3s": 8.2 },
  { "date": "2025-01-02", "flowM3s": 9.1 }
]
```

`scenarios/SC-001/maintenance.json`

```
[
  { "date": "2025-01-05", "label": "Restart turbine after trip" },
  { "date": "2025-01-09", "label": "Clear debris from intake" },
  { "date": "2025-01-12", "label": "Start annual inspection" },
  { "date": "2025-01-13", "label": "End inspection — restart" }
]
```

`scenarios/SC-001/losses.json` (answer key — NOT sent to the browser during gameplay)

```
[
  {
    "zoneId": 1,
    "startDate": "2025-01-04",
    "endDate": "2025-01-04",
    "category": "breakdown",
    "subcategory": "turbine_failure",
    "description": "Turbine trip — zero production despite normal flow of 12 m3/s",
    "expectedKwhPerDay": 8500,
    "actualKwh": 0,
    "expectedKwh": 8500,
    "lossSizeKwh": 8500,
    "revenueLossEur": 1190
  }
]
```

What Gets Sent to the Browser

During gameplay, the frontend receives:

- `config.json` — plant info and game settings
- `production.json` — full production series
- `rain.json` — full rain series
- `flow.json` — full flow series
- `indicators.json` — additional indicators (if any)
- `maintenance.json` — maintenance event dots (dates + labels)

NOT sent: `losses.json` — the answer key stays on the server until submission.

The player must identify loss areas themselves by drawing on the chart. There are no pre-marked loss zones visible during gameplay.

Bernard's Workflow

```
1. Pick a real plant's data (anonymize names)
        ↓
2. Film a short intro video of the plant (phone is fine)
        ↓
3. Export production CSV from Naia or from the operator's system
```

```
        ↓
4. Export rain CSV (from Meteo-France or Naia)
        ↓
5. Export flow CSV (from Hubeau or Naia)
        ↓
6. Open the production data in a spreadsheet
        ↓
7. Identify loss periods visually (days where production doesn't match flow/rain)
        ↓
8. For each loss: determine category, subcategory, and expected production
        ↓
9. Create the losses CSV: start_date, end_date, category, subcategory, expected_kwh_per_day, description
        ↓
10. Create maintenance events CSV: date, label (from operator logbook)
        ↓
11. Run the preparation tool
        ↓
12. Review: does the chart look right? Are the loss areas clear enough?
        ↓
13. Playtest: can someone find and diagnose these losses from the chart?
        ↓
14. Adjust if needed (make clues more obvious, add/remove losses or maintenance events)
```

Guidelines for Bernard

Making Good Scenarios

- **Aim for 5-8 loss areas per scenario** (Standard mode)
- **Mix easy and hard losses**: include 2-3 obvious ones (zero production + normal flow = breakdown) and 2-3 subtle ones (partial production reduction during high flow = constraint)
- **Use realistic patterns**: operators should recognize these as plausible situations
- **Vary the causes**: don't make 5 out of 8 losses the same category
- **Include at least one of each**: if possible, have each of the 6 categories represented
- **The context must tell the story**: if you mark a loss as "environmental", make sure the rain/flow data on those days actually shows a weather event
- **Maintenance events as clues**: place them strategically. A "restart turbine" dot the day after a zero-production period is a strong clue for "breakdown"
- `expected_kwh_per_day` **must be realistic**: use the average production of surrounding normal days as a reference

Common Pitfalls

- **Loss area not visible**: if the production bar on a loss day is still high relative to expected, the loss won't be obvious enough to draw. Make sure there's a clear visual gap.
- **Ambiguous causes**: some losses could reasonably be 2 categories. That's fine for the real world, but for a game, make sure there's a "best" answer. Use maintenance dots and the description to make it clear.
- **Too many losses**: >8 losses in a 5-minute game = players can't finish. Keep it manageable.
- **All easy or all hard**: mix it up. Start with obvious losses to build confidence, then challenge them.
- **No maintenance events near losses**: maintenance dots are the main contextual clue. Without them, some losses become impossible to classify.

Number of Scenarios Needed

Conference Duration	Minimum Scenarios	Ideal
Half day	3	5
Full day	5	8

Multi-day	8	10+

Players who replay get a different scenario. With 5 scenarios and random assignment, the chance of two adjacent players getting the same one is 20% — acceptable.

Scenario Validation Checklist

Before a scenario is considered ready:

- ☐ All CSVs load without errors in the preparation tool
- ☐ Intro video plays correctly (30-90s, reasonable quality)
- ☐ Chart looks correct: production, rain, and flow align visually
- ☐ Each loss area is visually identifiable on the chart (production clearly below expected)
- ☐ Context indicators (rain, flow) support the correct diagnosis
- ☐ Maintenance events give useful clues without being too obvious
- ☐ Subcategories are unambiguous given the available data
- ☐ `expected_kwh_per_day` values are realistic (comparable to surrounding normal days)
- ☐ Descriptions are clear and educational
- ☐ Revenue loss figures are plausible
- ☐ At least 2 team members can playtest and score >50%
- ☐ No accidental data that reveals the real plant identity

---

04 - Game Flow & UX

> ***All screens are in French.*** *The target audience is French-speaking hydropower operators. All UI labels, buttons, and messages shown below are in French.*

Screen Flow

```
[Bienvenue] → [Vidéo intro] → [GO!] → [Jeu] → [Résumé] → [Saisir email] → [Classement]
     ↑                                                                          |
     └──────────────────────── [Nouvelle partie] ←──────────────────────────────┘
```

Key change vs. classic arcade flow: Registration (email) happens **after** the game, not before. The player is hooked by the game first, then motivated to leave their email to receive detailed results by email. This increases conversion. Detailed results are **only sent by email** — they are not shown on screen (keeps the flow fast for the next player).

Screen 1: Bienvenue (Welcome)

Purpose: Attract passers-by.

```
┌──────────────────────────────────────────────┐
│                                              │
│          [Logo Naia]                         │
│                                              │
│   Détectez les pertes cachées !              │
│                                              │
│   Les exploitants hydroélectriques perdent   │
│   des milliers d'euros sans le savoir.       │
│   Saurez-vous trouver et diagnostiquer      │
│   les pertes de production ?                 │
```

```
│                                          │
│          [ JOUER ]                       │
│                                          │
│   ─── Classement ───                     │
│   1. Pierre M.  ·····  142 pts           │
│   2. Sophie L.  ·····  128 pts           │
│   3. Marc D.    ·····  115 pts           │
│                                          │
│   Aujourd'hui : 47 joueurs | Moy. : 89   │
│                                          │
└──────────────────────────────────────────┘
```

- Auto-returns here after 60s of inactivity
- Top 5 visible for competitive pull

Screen 2: Vidéo d'introduction

```
┌──────────────────────────────────────────┐
│                                          │
│   ┌──────────────────────────────────┐   │
│   │                                  │   │
│   │     [Vidéo de la centrale]       │   │
│   │                                  │   │
│   │     Centrale de Moulins          │   │
│   │     450 kW — La Dordogne, France │   │
│   │                                  │   │
│   └──────────────────────────────────┘   │
│                                          │
│   Découvrez cette centrale avant de jouer. │
│                                          │
│          [ C'EST PARTI ! ]  (ou attendre) │
│                                          │
└──────────────────────────────────────────┘
```

- **Video**: Local MP4/WebM file, configured per scenario
- Player can press **C'EST PARTI !** anytime (even during video) to skip ahead
- If no button pressed, game starts automatically when video ends
- Plant name, capacity, and river shown as overlay text on the video or below it
- Purpose: Gives the player context about the plant they're about to analyze

Screen 3: Décompte (Countdown)

```
        3... 2... 1... C'EST PARTI !
```

Full-screen, 3 seconds. Builds anticipation.

Screen 4: Jeu (Gameplay — Core)

This screen is 100% identical to the real Naia platform dashboard. The game doubles as a live product demo — what the player sees IS Naia. The only addition is a **chronometer** (countdown timer) displayed at the top.

The player must: (1) draw expected production lines to identify losses, (2) classify each loss by category and subcategory.

```
┌──────────────────────────────────────────────────────────────────────┐
│ ⏱ 4:32          Score : 35           Pertes trouvées : 2             │
├──────────────────────────────────────────────────────────────────────┤
```

```
│        │                                                    │
│ PANEL  │  ZONE GRAPHIQUE                                    │
│        │                                                    │
│ Indica-│  ▼▼  ▼▼▼  ▼    ▼▼▼▼  ▼▼   ▼▼  ▼▼▼     ← Pluie       │
│ teurs  │  ──────────────────────────────────                 │
│        │  ~~  ~~~  ~~~~  ~~  ~~~  ~~~~  ~~  ~     ← Débit       │
│ ☐ Débit│  ──────────────────────────────────                 │
│   stn  │  ██ ██ ██ ▒▒ ██ ██ ▓▓ ██ ██ ██ ▒▒ ██    ← Production  │
│   B    │          • ← point maintenance                      │
│ ☐ Débit│           ---- ← ligne tracée par le joueur          │
│   stn  │                                                     │
│   C    │  [ MODE DESSIN : Cliquez pour tracer la production     │
│        │    attendue ]                                          │
│ ☐ Comp-│  ── Vos pertes ──                                    │
│   ara- │  • Perte 1 : Panne > Défaillance turbine  [35 pts]    │
│   teur │  • Perte 2 : [en cours...] cliquez pour définir        │
│   (CHE │                                                     │
│   aval)│                                                     │
│        │              [ VALIDER ]                            │
│        │                                                     │
└────────────────────────────────────────────────────────────┘
```

Chronometer (Top Bar)

- **Timer**: Counts down from 5:00. Orange at 1:00, red + pulse at 0:15
- **Score**: Running total (updates after each loss is classified)
- **Progress**: "Pertes trouvées : 2" — how many losses the player has identified
- This is the ONLY element not present in the real Naia dashboard

Left Panel: Sélecteur d'indicateurs

The left panel lists **additional data sources** the player can toggle on/off to overlay on the chart. This mirrors the real Naia dashboard where operators configure which context indicators to display.

Default (always visible):

- Pluie (station principale) — inverted bars
- Débit (station principale) — line
- Points maintenance — always shown

Available to add (checkboxes):

- Additional flow stations (e.g., "Station B — amont", "Station C — affluent")
- Comparator: production of another plant (e.g., "CHE Le Moulin — aval")
- Comparator: production of the same plant in a previous year
- Additional rain station

How this helps the player:

- If a loss area shows low production but the primary flow station looks normal, adding a **second flow station upstream** might reveal that the water was diverted
- A **comparator plant** showing normal production on the same day proves the water was there → the loss is plant-specific (breakdown, not environmental)
- A **previous year comparator** helps distinguish seasonal patterns from real losses

Strategic value:

- Players who use additional indicators can make better diagnoses
- This mirrors exactly how the real Naia tool works → strongest possible product demo
- Time spent adding indicators = time not spent finding losses → risk/reward tradeoff

Points maintenance

Visible as small dots on the production line at specific dates:

- **Hover** or **click** a dot → popup with the maintenance label
- Example: "Redémarrage turbine après déclenchement", "Nettoyage grille", "Remplacement capteur de débit"
- These give strong contextual clues for diagnosis
- Some dots are near loss areas (helpful), some are routine operations (not near losses)

```
┌──────────────────────────┐
│  🔵 5 jan.                 │
│  "Redémarrage turbine après  │
│   déclenchement"           │
└──────────────────────────┘
```

Drawing Interaction

The player draws expected production lines on the chart to mark loss areas:

1. **Enter draw mode**: Click the "Dessiner" button or click directly on the chart
2. **Click on start date**: First click sets the start of the loss area
3. **Click on end date**: Second click sets the end of the loss area
4. **Draw expected line**: Between start and end, the player clicks/drags to set the expected production level for each day (or a single flat line)
5. **Loss area appears**: The gap between the drawn line and actual production is shaded — this is the loss the player identified
6. **Cause selector opens**: Player picks category + subcategory

Simplified drawing option: For speed, the player can:

- Click a start date and end date
- The system draws a flat line at the average of surrounding normal days
- Player adjusts the line height if needed (drag up/down)

Sélection de cause (After Drawing a Loss Area)

When the player finishes drawing a loss area:

```
┌──────────────────────────────────┐
│  Perte 3 : 15-16 jan.            │
│  Perte estimée : 15 200 kWh      │
│                                   │
│  Quelle est la cause ?           │
│                                   │
│  Catégorie :                     │
│  ○ 🔧 Panne                      │
│  ○ 📅 Maintenance                 │
│  ○ 🌿 Environnement              │
│  ○ ⚙ Exploitation                 │
│  ○ ⚡ Réseau / Contrainte         │
│  ○ ⚠ Problème données            │
│                                   │
│  Sous-catégorie :                │
│  [▼ Choisir la sous-catégorie...]  │
│  (apparaît après le choix)       │
│                                   │
│  [ CONFIRMER ]    [ SUPPRIMER ]   │
└──────────────────────────────────┘
```

Flow:

1. Player selects a **category** (6 radio buttons)
2. **Subcategory dropdown** appears with 3-5 options for that category
3. Player selects subcategory
4. **CONFIRMER** → score updates immediately:
   - Category: green flash (+10) or red flash (-5)
   - Subcategory: green flash (+5) or red flash (-2)
   - Size accuracy: shown as a bar (0-10 pts) — "Taille : 92% précision → 6 pts"
5. **SUPPRIMER** → removes the drawn area (if player changes their mind)

Changing an answer: Click an existing loss area on the chart → reopens the selector. Previous points reversed, new points applied.

Liste des pertes (Bottom of Chart Area)

Shows all losses the player has identified:

```
• Perte 1 : ✅ Panne > Défaillance turbine — 10 + 5 + 8 = 23 pts
• Perte 2 : ❌ Environnement > Crue — 10 + (-2) + 6 = 14 pts
• Perte 3 : [dessin en cours...]
```

Clicking a row highlights the corresponding area on the chart.

Valider (Submit)

- Available anytime — player decides when they're done
- Shows time comparison: "Temps utilisé : 2:47. Temps normal : 3:00 → 13 secondes plus rapide = +13 points bonus"
- Confirmation required

Timer Expiry

- At 0:00, game ends automatically
- All drawn losses are scored; unfinished drawing is discarded
- Transition to results

Screen 5: Résumé (Summary Result)

A quick score overview — shown immediately after gameplay ends. Keeps the energy up.

```
┌──────────────────────────────────────────┐
│                                          │
│   🎯 Votre score : 142 points            │
│   📊 Classement : #7 sur 42 joueurs      │
│                                          │
│   Pertes trouvées : 5/7                  │
│   Bonus temps : +30 pts                  │
│                                          │
│   Voulez-vous recevoir vos résultats     │
│   détaillés par email ?                  │
│                                          │
│          [ OUI, VOIR MES RÉSULTATS → ]   │
│                                          │
│          [ NON, VOIR LE CLASSEMENT ]     │
│                                          │
└──────────────────────────────────────────┘
```

If player clicks "NON" → skip to Classement screen. If "OUI" → go to email input.

Screen 6: Saisir email (Email Input)

On-screen keyboard — there is no physical keyboard at the arcade cabinet. The player uses the mouse to type their email on a virtual keyboard.

```
┌──────────────────────────────────────────┐
│                                          │
│   Entrez votre adresse email             │
│                                          │
│   ┌──────────────────────────────────┐   │
│   │ pierre.martin@edf.fr             │   │
│   └──────────────────────────────────┘   │
│                                          │
│   ┌───┬───┬───┬───┬───┬───┬───┬───┬───┬───┐ │
│   │ 1 │ 2 │ 3 │ 4 │ 5 │ 6 │ 7 │ 8 │ 9 │ 0 │ │
│   ├───┼───┼───┼───┼───┼───┼───┼───┼───┼───┤ │
│   │ a │ z │ e │ r │ t │ y │ u │ i │ o │ p │ │
│   ├───┼───┼───┼───┼───┼───┼───┼───┼───┼───┤ │
│   │ q │ s │ d │ f │ g │ h │ j │ k │ l │ m │ │
│   ├───┼───┼───┼───┼───┼───┼───┼───┼───┼───┤ │
│   │ w │ x │ c │ v │ b │ n │ . │ - │ _ │ ⌫ │ │
│   ├───┴───┴───┴───┴───┴───┴───┴───┴───┴───┤ │
│   │    @      │   .com    │   .fr         │ │
│   └───────────┴───────────┴───────────────┘ │
│                                          │
│   ☐ J'accepte de recevoir mes résultats  │
│     par email                            │
│   ☐ Je souhaite en savoir plus sur Naia  │
│                                          │
│   Prénom *  [_______________]            │
│   Nom *     [_______________]            │
│   Entreprise [______________]            │
│                                          │
│         [ ENVOYER ET VOIR LE CLASSEMENT → ]  │
│                                          │
└──────────────────────────────────────────┘
```

On-screen keyboard features:

- AZERTY layout (French keyboard)
- Quick-insert buttons for `@`, `.com`, `.fr` (most common email domains)
- Backspace key (⌫)
- Large click targets (44px+ for mouse precision)
- Email validation: check format before proceeding
- Basic name fields (first name, last name) also use on-screen keyboard
- Optional company field

After submitting, the detailed results are **sent by email only** (not shown on screen). This keeps the flow fast — the screen transitions directly to the Classement. The email includes the full score breakdown, CTA to naia.energy/signup, and revenue impact figures.

Screen 7: Classement (Leaderboard — Second Browser Window, Same PC)

Displayed in a second browser window on a connected TV/monitor.

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
```

```
│   [Logo Naia]    DÉFI DÉTECTION DE PERTES                │
│                                                           │
│   🏆 CLASSEMENT                                          │
│   ──────────────────────────────────────────            │
│   #1  Pierre M.     EDF Hydro            142             │
│   #2  Sophie L.     CNR                  128             │
│   #3  Marc D.       Hydrovolt            115             │
│   ...                                                     │
│   #10 Emma S.       Indépendant           72             │
│   ──────────────────────────────────────────            │
│                                                           │
│   Aujourd'hui : 47 joueurs | Moy. : 89 | Record : 142   │
│                                                           │
│   🎮 En jeu : Marie D. — 3:42 restantes                  │
│                                                           │
└───────────────────────────────────────────────────────────┘
```

- Both game and scoreboard served from `localhost:3000` (same Nuxt server)
- Top 10 displayed
- Updates automatically (polling `localhost` — no SSE needed since it's local)
- Alternates with attract-mode animation when idle

Naia Signup Page (naia.energy/signup)

This page is part of this project's scope and lives on the naia.energy website. It is the landing page for the CTA button shown in game results and in the results email.

```
┌───────────────────────────────────────────────────────────┐
│                                                           │
│   [Logo Naia]                                             │
│                                                           │
│   Vous avez détecté des pertes comme un pro.              │
│   Naia le fait automatiquement pour votre centrale.       │
│                                                           │
│   ── Ce que Naia fait pour vous ──                        │
│   ✅ Détection automatique des pertes de production       │
│   ✅ Classification par catégorie et cause                │
│   ✅ Estimation du manque à gagner en €                   │
│   ✅ Alertes en temps réel                                │
│                                                           │
│   ── Essayez gratuitement ──                              │
│                                                           │
│   Prénom *      [_______________]                         │
│   Nom *         [_______________]                         │
│   Email *       [_______________]                         │
│   Entreprise    [_______________]                         │
│   Téléphone     [_______________]                         │
│                                                           │
│          [ CRÉER MON COMPTE GRATUIT ]                     │
│                                                           │
│   Déjà client ? Se connecter                              │
│                                                           │
└───────────────────────────────────────────────────────────┘
```

- **URL**: `https://naia.energy/signup` (or `/inscription`)

- **UTM tracking**: Links from the game include `?utm_source=conference-game&utm_campaign={conference_id}` to track conversions
- **Pre-fill**: If the player clicked the CTA from the results email, pre-fill email from the link parameter
- **Design**: Same Naia branding, responsive, works on mobile (players may scan a QR code from the email)
- **Backend**: Creates a lead in the Naia platform (or CRM)

Interaction Design

Mouse-Only — No Keyboard

The game runs on an arcade cabinet without a physical keyboard. All interaction is mouse-based:

- **Drawing**: Click to set points, drag optional for fine-tuning
- **Text input**: On-screen virtual keyboard (AZERTY) for email and name entry
- **Large click targets**: 44px+ for all interactive elements
- **Hover states**: Maintenance dots, chart bars, panel checkboxes
- **Clear cursor changes**: Crosshair in draw mode, pointer on interactive elements

Drawing UX Details

- Visual feedback: as the player draws, the loss area fills with a semi-transparent color
- The estimated loss size (kWh) updates in real-time as the line is adjusted
- Undo: click "Supprimer" in the cause selector, or right-click on a drawn area
- Maximum ~12 loss areas per game (prevent spam-drawing)

Panel Indicator Toggling

- Click checkbox → indicator appears/disappears on chart with smooth transition
- Each indicator has a color legend entry
- Maximum 3 additional indicators visible at once (chart readability)

Edge Cases

Situation	Behavior
Player walks away mid-game	Auto-timeout after 30s no interaction → save partial score → bienvenue
Player draws overlapping loss areas	Merge into one, or reject and show "Chevauchement avec Perte 2"
Player draws a loss where there isn't one	-5 points (false loss) — shown in results
Timer expires during drawing	Unfinished drawing discarded, completed losses scored
Timer expires during popup	Popup closes, current selection NOT saved, game ends
Player wants to replay	Gets a different scenario. Best score counts for leaderboard
Player skips intro video	Allowed — press C'EST PARTI ! anytime
Intro video file missing	Skip video screen, go straight to countdown
Internet unavailable	Game works 100% locally. Email queued, sent when connection returns
Player skips email input	Score still saved in leaderboard. No email sent.

---

05 - Registration, Scoreboard & Email

Registration (Post-Game)

Registration happens **after** the game, not before. The player plays first (no friction), then enters their email to see detailed results. This maximizes both play rate and lead capture quality — players who leave their email are genuinely interested.

Data Model (SQLite — Local)

```sql
CREATE TABLE game_participants (
    id                TEXT PRIMARY KEY,  -- UUID
    first_name        TEXT,
    last_name         TEXT,
    email             TEXT,
    company           TEXT,
    role              TEXT,              -- plant_operator, owner, engineer, consultant, student, other
    email_consent     INTEGER DEFAULT 1, -- boolean
    marketing_consent INTEGER DEFAULT 0, -- boolean
    conference_id     TEXT NOT NULL,      -- e.g., "hydro-2025-grenoble"
    created_at        TEXT DEFAULT (datetime('now'))
);

CREATE TABLE game_sessions (
    id                TEXT PRIMARY KEY,  -- UUID
    participant_id    TEXT REFERENCES game_participants(id),  -- NULL until email entered
    scenario_id       TEXT NOT NULL,
    started_at        TEXT NOT NULL,
    ended_at          TEXT,
    time_limit_sec    INTEGER DEFAULT 300,
    time_used_sec     INTEGER,
    status            TEXT DEFAULT 'in_progress',  -- in_progress, completed, abandoned

    -- Scoring
    total_score       INTEGER DEFAULT 0,
    losses_found      INTEGER DEFAULT 0,
    losses_correct    INTEGER DEFAULT 0,
    false_losses      INTEGER DEFAULT 0,
    missed_losses     INTEGER DEFAULT 0,
    category_points   INTEGER DEFAULT 0,
    subcategory_points INTEGER DEFAULT 0,
    size_points       INTEGER DEFAULT 0,
    time_bonus        INTEGER DEFAULT 0,  -- positive = faster, negative = slower than normal
    revenue_found     REAL DEFAULT 0,
    revenue_missed    REAL DEFAULT 0,

    -- Display name (entered post-game or auto-generated)
    display_name      TEXT,  -- "Pierre M." for leaderboard

    -- Sync tracking
    synced_to_sheets  INTEGER DEFAULT 0, -- boolean
    sync_status       TEXT DEFAULT 'pending',  -- pending, unverified, verified
    synced_at         TEXT,
    console_id        TEXT,              -- C1 or C2

    created_at        TEXT DEFAULT (datetime('now'))
);

CREATE TABLE game_selections (
    id                  TEXT PRIMARY KEY,  -- UUID
    session_id          TEXT NOT NULL REFERENCES game_sessions(id),
```

```sql
    zone_id             INTEGER,           -- NULL if false loss (no matching answer key zone)
    start_date          TEXT NOT NULL,
    end_date            TEXT NOT NULL,
    drawn_expected_kwh  TEXT,              -- JSON array of daily expected values drawn by player
    loss_size_kwh       REAL,              -- calculated from drawn line
    selected_category   TEXT NOT NULL,
    selected_subcategory TEXT NOT NULL,
    is_correct_category INTEGER,           -- boolean, set after submission
    is_correct_subcategory INTEGER,        -- boolean, set after submission
    size_accuracy       REAL,              -- 0.0 to 1.0, set after submission
    category_points     INTEGER,
    subcategory_points  INTEGER,
    size_points         INTEGER,
    is_false_loss       INTEGER DEFAULT 0, -- boolean: player drew a loss where there isn't one
    selected_at         TEXT DEFAULT (datetime('now'))
);
```

Flow

1. Player presses JOUER → game starts (no registration)
2. Game session created with `participant_id = NULL`
3. After game → summary screen → player enters email + name
4. Participant record created, linked to session
5. Detailed results shown + email sent
6. If player skips email → session saved with `participant_id = NULL`, score still on leaderboard as "Joueur anonyme"

Returning Players

- Same email → same participant record, new session
- Different scenario assigned (round-robin from available scenarios excluding already-played ones)
- Leaderboard shows **best score** across all sessions

GDPR

Requirement	Implementation
Consent for results email	Explicit checkbox (pre-checked)
Consent for marketing	Separate checkbox, unchecked by default
Data minimization	Name, email, company — nothing more
Right to erasure	Admin can delete by email
Retention	Auto-delete 90 days after conference
Privacy notice	Link on email input form
Data location	Local SQLite on game PC + Google Sheets (EU)

Classement (Leaderboard)

Architecture: Same PC, Second Window

The leaderboard runs as a second browser window on the game PC, connected to a TV/monitor via HDMI:

```
Game PC
├── Chrome Window 1 (primary screen): http://localhost:3000 → game
└── Chrome Window 2 (secondary screen): http://localhost:3000/classement → leaderboard
```

Both windows are served by the same local Nuxt server. No network, no SSE, no separate computer.

Scoreboard Data

```
GET /api/scoreboard?conference_id=hydro-2025-grenoble

{
  "conferenceId": "hydro-2025-grenoble",
  "stats": {
    "totalPlayers": 47,
    "averageScore": 89,
    "highestScore": 142,
    "totalLossesFound": 312,
    "totalRevenueDiagnosed": 387420
  },
  "leaderboard": [
    {
      "rank": 1,
      "displayName": "Pierre M.",
      "company": "EDF Hydro",
      "score": 142,
      "lossesFound": 7,
      "playedAt": "2025-06-15T10:15:00Z"
    }
  ],
  "currentGame": {
    "playerName": "Marie D.",
    "timeRemaining": 222,
    "lossesFound": 2,
    "status": "playing"
  }
}
```

Real-Time Updates

Since both windows run on the same machine:

- Scoreboard page polls `localhost:3000/api/scoreboard` every 3 seconds
- Sub-millisecond response time (local SQLite query)
- No SSE or WebSocket needed — polling is simpler and sufficient for localhost

Scoreboard Display

- Full-screen page at `/classement?conference=hydro-2025-grenoble`
- Top 10 with animated entry when a new score enters the ranking
- Chrome fullscreen (F11) on the TV
- No interaction needed — auto-updates
- All labels in French: "Classement", "Joueurs aujourd'hui", "Moyenne", "Record", "En jeu"

Google Sheets Sync (Shared Leaderboard + Backup)

Purpose

- **Shared leaderboard**: When running 2 consoles, Google Sheets is the single source of truth for the combined Classement
- **Backup**: If a game PC fails, all data is in the cloud
- **Remote monitoring**: Check scores from a phone or other device
- **CRM export**: Sheets is already in a format marketers can use

How It Works

After each completed game session, the server appends a row to a Google Sheet and **verifies** it was written correctly:

```
| Session ID | Console | Timestamp | First Name | Last Name | Email | Company | Score | Losses Found | Scenario |
Marketing Consent |
|-----------|---------|-----------|-----------|-----------|-------|---------|-------|-------------|----------|---
----------------|
| abc-123   | C1      | 2025-06-15 10:15 | Pierre | Martin | pierre@edf.fr | EDF Hydro | 142 | 7 | SC-001 | Oui |
```

Sync with Write Verification

Conference WiFi is unreliable. The sync must be **robust** — we cannot silently lose scores. The process:

```
1. WRITE: Append row to Google Sheet
         ↓
2. VERIFY: Read back the last rows, check if our session_id appears
         ↓
   ┌── Found? ──┐
   │ YES        │ NO
   │            │
   ▼            ▼
3a. Mark as       3b. Mark as UNVERIFIED
    synced_to_       in SQLite, retry
    sheets = 1       in 30 seconds
```

```
// server/services/sheets-sync.ts

async function syncAndVerify(session: GameSession, participant: Participant) {
  try {
    // Step 1: Write
    await sheets.spreadsheets.values.append({
      spreadsheetId: SHEETS_ID,
      range: 'Results!A:K',
      valueInputOption: 'USER_ENTERED',
      requestBody: {
        values: [[
          session.id,
          process.env.CONSOLE_ID || 'C1',  // identifies which console
          new Date().toISOString(),
          participant.firstName,
          participant.lastName,
          participant.email,
          participant.company,
          session.totalScore,
          session.lossesFound,
          session.scenarioId,
          participant.marketingConsent ? 'Oui' : 'Non'
        ]]
      }
    })

    // Step 2: Verify — read back last 20 rows, check our session_id
    const response = await sheets.spreadsheets.values.get({
      spreadsheetId: SHEETS_ID,
      range: 'Results!A:A',  // just the session_id column
```

```
    })
    const allIds = response.data.values?.flat() || []

    if (allIds.includes(session.id)) {
      // Step 3a: Verified — mark as synced
      markAsSynced(session.id, 'verified')
    } else {
      // Step 3b: Write succeeded but verification failed — retry later
      markAsSynced(session.id, 'unverified')
    }
  } catch (error) {
    // Network error — stay in queue, retry later
    console.warn('Sheets sync failed, will retry:', error.message)
  }
}

// Background retry: every 30 seconds, retry unsynced + unverified rows
setInterval(async () => {
  const pending = db.select().from(gameSessions)
    .where(
      and(
        eq(gameSessions.status, 'completed'),
        or(
          eq(gameSessions.syncedToSheets, 0),        // never synced
          eq(gameSessions.syncStatus, 'unverified')   // synced but not verified
        )
      )
    )
    .all()

  for (const session of pending) {
    await syncAndVerify(session, getParticipant(session.participantId))
  }
}, 30_000)
```

Classement: Reading from Google Sheets

When running **2 consoles**, the Classement reads from Google Sheets (not local SQLite) to show the **combined** leaderboard:

```
// server/api/scoreboard.get.ts

async function getLeaderboard(conferenceId: string) {
  // Try Google Sheets first (combined leaderboard)
  try {
    const response = await sheets.spreadsheets.values.get({
      spreadsheetId: SHEETS_ID,
      range: 'Results!A:K',
    })
    // Parse rows, sort by score descending, return top 10
    return buildLeaderboardFromSheets(response.data.values)
  } catch (error) {
    // Fallback: local SQLite only (this console's scores)
    return buildLeaderboardFromSQLite(conferenceId)
  }
}
```

The Classement page polls every 5 seconds. When WiFi is available → combined leaderboard from both consoles. When WiFi is down → local leaderboard only (with a small indicator "⚠ Mode local").

Sync Status in Admin Panel

The admin panel shows sync health per session:

Status	Meaning	Action
✅ `verified`	Written to Sheets AND confirmed	None
⏳ `unverified`	Written but not yet confirmed	Auto-retry every 30s
❌ `pending`	Not yet written (no internet)	Auto-retry every 30s
🔄 `retrying`	Currently retrying	Wait

Staff can also trigger a **Force Sync** from the admin panel to immediately retry all pending rows.

Setup

1. Create a Google Sheet (shared with team)
2. Create a service account in Google Cloud Console
3. Share the Sheet with the service account email
4. Place the JSON key on **both** game PCs
5. Set in `.env` on each PC:
   - `GOOGLE_SHEETS_ID` — same sheet ID on both
   - `GOOGLE_CREDENTIALS_PATH` — path to service account JSON
   - `CONSOLE_ID` — `C1` on console 1, `C2` on console 2
   - `MULTI_CONSOLE=true` — enables Sheets-based leaderboard reading

Results Email (via Infomaniak SMTP)

Trigger

- Sent after the player enters their email and clicks "VOIR MES RÉSULTATS"
- Via Infomaniak SMTP (`mail.infomaniak.com:587` with STARTTLS)
- Uses Nodemailer — standard Node.js SMTP library
- If offline: queued in SQLite, sent when connection returns
- Sent from: `game@naia.energy`
- Reply-to: `contact@naia.energy`

Email Configuration

```
// server/services/email.ts
import nodemailer from 'nodemailer'

const transporter = nodemailer.createTransport({
  host: 'mail.infomaniak.com',
  port: 587,
  secure: false, // STARTTLS
  auth: {
    user: process.env.SMTP_USER,     // game@naia.energy
    pass: process.env.SMTP_PASSWORD,
  },
})
```

Content

Subject: Vos résultats Naia — Score : {score} pts 🎯

Body (HTML template with plain text fallback — nicely formatted):

```
Bonjour {firstName},

Merci d'avoir relevé le Défi Détection de Pertes Naia !

VOS RÉSULTATS
━━━━━━━━━━━━

Score :      {totalScore} points
Classement : #{rank} sur {totalPlayers} joueurs
Centrale :   {plantName} ({capacityKw} kW)

PERTES IDENTIFIÉES
━━━━━━━━━━━━━━━━━

{for each loss}
✅ Perte {n} ({dates}) : {category} > {subcategory}
   Catégorie : ✅ +10 | Sous-catégorie : ✅ +5 | Taille : 95% → 8 pts
   « {description} »

❌ Perte {n} ({dates}) : Vous avez dit {playerCategory} > {playerSubcategory}
   Correct : {correctCategory} > {correctSubcategory}
   Catégorie : ❌ -5 | Sous-catégorie : ❌ -2 | Taille : 82% → 1 pt
   « {description} »

⬜ Manquée ({dates}) : {correctCategory} > {correctSubcategory}
   « {description} »
{end for}

Bonus temps : {timeBonus} ({timeDiff} secondes par rapport au temps normal)

IMPACT FINANCIER
━━━━━━━━━━━━━━━

Pertes correctement identifiées : {revenueFound} €
Pertes totales de la centrale : {totalRevenueLoss} € / an

━━━━━━━━━━━━━━━━━
```

Features

- **View participants**: table with name, email, company, score, scenario
- **Export CSV**: download all participants for CRM import
- **Game stats**: total games, average score, completion rate, most-found losses
- **Reset session**: clear a stuck game
- **Scenario management**: enable/disable scenarios
- **Sync status**: show which rows have been synced to Google Sheets
- **Force sync**: manually trigger Google Sheets sync

Access

- URL: `/admin?key={admin_secret}`
- Single shared secret (not a full auth system)
- Mobile-friendly for staff use on phones (over local WiFi or internet)

CSV Export Format

```
first_name,last_name,email,company,score,losses_found,rank,scenario,played_at,marketing_consent
Pierre,Martin,pierre@example.com,EDF Hydro,142,7,1,SC-001,2025-06-15T10:15:00Z,true
Sophie,Laurent,sophie@cnr.fr,CNR,128,6,2,SC-003,2025-06-15T11:30:00Z,false
```

Ready for import into HubSpot, Pipedrive, or similar CRM.

---

06 - Technical Implementation

Architecture

Single Nuxt 3 application running **locally** on the game PC — frontend and backend in one project. No cloud dependency during gameplay.

```
┌──────────────────────────────────────────────────────────┐
│              Game PC                                     │
│                                                          │
│  Chrome Window 1 (game)     Chrome Window 2 (scoreboard) │
│  localhost:3000             localhost:3000/scoreboard     |
│  Primary screen             TV / secondary screen        │
│         │                         │                      │
│         │ REST API                │ Polling (3s)         │
│         ▼                         ▼                      │
│  ┌──────────────────────────────────────────┐          |
│  │        Nuxt 3 (Nitro server)              │          │
│  │                                           │          │
│  │  server/api/register.post.ts              │          │
│  │  server/api/start.post.ts                 │          │
│  │  server/api/submit.post.ts                │          │
│  │  server/api/results/[id].get.ts           │          │
│  │  server/api/scoreboard.get.ts             │          │
│  │  server/api/admin/*                       │          │
│  │                                           │          │
│  │  SQLite (game.db)       Scenarios (JSON+MP4)  │          │
│  │  participants           scenarios/SC-001/     │          │
│  │  sessions                 config.json         │          │
│  │  selections               production.json      │          │
│  │                           losses.json         │          │
│  │                           maintenance.json    │          │
```

```
│  │                          intro.mp4          │       │
│  └──────────────────────────────────────────────┘       │
│         │                                                │
│         │ Background sync (when internet available)      │
│         ▼                                                │
│  Google Sheets API → cloud backup + remote monitoring    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Tech Stack

Layer	Technology	Rationale
Framework	Nuxt 3 (Vue 3 + TypeScript)	Same as Naia — consistent look & feel
UI Library	Nuxt UI	Same as Naia
Charts	uPlot	Same as Naia — fast time-series rendering
Drawing	HTML5 Canvas overlay	Custom drawing layer on top of uPlot chart
State	Pinia	Same as Naia
Server	Nitro (built into Nuxt)	No separate backend needed
Database	SQLite via better-sqlite3	Zero-config, file-based, local
ORM	Drizzle ORM	Lightweight, TypeScript-native
Cloud sync	Google Sheets API v4	Background backup, remote monitoring
Email	Infomaniak SMTP via Nodemailer	Send results when internet available
Video	HTML5 `<video>`	Local MP4/WebM files, no streaming

Key Data Structures

Scenario Data (JSON files)

`config.json` — sent to browser

```typescript
interface ScenarioConfig {
  id: string                    // "SC-001"
  plantName: string             // "Centrale de Moulins"
  capacityKw: number            // 450
  river: string                 // "La Dordogne"
  country: string               // "France"
  periodStart: string           // "2025-01-01"
  periodEnd: string             // "2025-02-28"
  difficulty: 'easy' | 'standard' | 'hard'
  timeLimitSec: number          // 300
  normalCompletionTimeSec: number // 180 — expected time for a competent operator
  numLosses: number             // 6
  introVideo: string            // "intro.mp4"
}
```

`production.json` — sent to browser

```typescript
interface ProductionDay {
  date: string                  // "2025-01-01"
  energyKwh: number             // 8540
}
```

`rain.json` — sent to browser

```typescript
interface RainDay {
  date: string
  rainMm: number
}
```

`flow.json` — sent to browser

```typescript
interface FlowDay {
  date: string
  flowM3s: number
}
```

`maintenance.json` — sent to browser (contextual clues)

```typescript
interface MaintenanceEvent {
  date: string                  // "2025-01-05"
  label: string                 // "Restart turbine after trip"
}
```

`indicators.json` — additional indicators available in the panel

```typescript
interface AdditionalIndicator {
  id: string                    // "flow-station-b"
  label: string                 // "Station B — upstream"
  type: 'flow' | 'rain' | 'comparator'
  unit: string                  // "m3/s" or "kWh" or "mm"
  data: Array<{ date: string; value: number }>
}
```

`losses.json` — answer key, NEVER sent to browser during gameplay

```typescript
interface LossZone {
  zoneId: number                // 1, 2, 3...
  startDate: string
  endDate: string
  category: LossCategory
  subcategory: string           // e.g., "turbine_failure"
  description: string           // Bernard's explanation
  expectedKwhPerDay: number     // What production should have been
  actualKwh: number             // Sum of actual production in the period
  expectedKwh: number           // Sum of expected production in the period
  lossSizeKwh: number           // expectedKwh - actualKwh
  revenueLossEur: number
}
```

```
type LossCategory = 'breakdown' | 'maintenance' | 'environmental'
  | 'operational' | 'grid_constraint' | 'data_issue'
```

Game State (Pinia Store)

```
interface GameState {
  // Session
  participantId: string | null
  sessionId: string | null
  scenario: ScenarioConfig | null

  // Game flow (register happens AFTER gameplay, detailed results only by email)
  status: 'idle' | 'video' | 'countdown' | 'playing' | 'submitting' | 'summary' | 'email_input' | 'classement'

  // Timer
  timeRemaining: number         // seconds
  timeLimitSec: number

  // Chart data
  production: ProductionDay[]
  rain: RainDay[]
  flow: FlowDay[]
  maintenanceEvents: MaintenanceEvent[]
  additionalIndicators: AdditionalIndicator[]
  activeIndicators: string[]    // IDs of toggled-on indicators

  // Drawing state
  drawingMode: boolean
  currentDrawing: DrawnLoss | null

  // Player's identified losses
  drawnLosses: DrawnLoss[]

  // Results (populated after submit)
  results: GameResults | null
}

interface DrawnLoss {
  id: string                    // client-generated UUID
  startDate: string
  endDate: string
  expectedKwhPerDay: number[]   // one value per day in the range (drawn line heights)
  lossSizeKwh: number           // calculated: sum(expected - actual) for each day
  category: LossCategory | null
  subcategory: string | null
  confirmed: boolean            // has the player confirmed category + subcategory?
}

interface GameResults {
  totalScore: number
  rank: number
  totalPlayers: number
  categoryPoints: number
  subcategoryPoints: number
  sizePoints: number
```

```
  timeBonus: number             // positive = faster, negative = slower
  revenueFound: number
  revenueMissed: number
  totalRevenueLoss: number
  breakdown: Array<{
    type: 'correct' | 'wrong_category' | 'wrong_subcategory' | 'missed' | 'false_loss'
    playerLoss: DrawnLoss | null
    answerKeyLoss: LossZone | null
    categoryPoints: number
    subcategoryPoints: number
    sizePoints: number
    sizeAccuracy: number
    totalPoints: number
  }>
}
```

API Endpoints

`POST /api/register`

```
// Request
{ firstName: string, lastName: string, email: string, company?: string,
  role?: string, emailConsent: boolean, marketingConsent: boolean,
  conferenceId: string }

// Response
{ participantId: string }
```

`POST /api/start`

```
// Request
{ participantId: string }

// Response — includes scenario data but NOT the answer key
{
  sessionId: string,
  scenario: ScenarioConfig,
  production: ProductionDay[],
  rain: RainDay[],
  flow: FlowDay[],
  maintenanceEvents: MaintenanceEvent[],
  additionalIndicators: AdditionalIndicator[]
  // NO losses — player must find them!
}
```

`POST /api/submit`

```
// Request
{
  sessionId: string,
  drawnLosses: Array<{
    startDate: string,
```

```
    endDate: string,
    expectedKwhPerDay: number[],
    lossSizeKwh: number,
    category: LossCategory,
    subcategory: string
  }>,
  timeUsedSec: number
}

// Response — full results including correct answers
{ results: GameResults }
```

`GET /api/results/:id`

Returns same `GameResults` for a completed session.

`GET /api/scoreboard?conferenceId=xxx`

Returns leaderboard + stats. Polled every 3s by scoreboard window.

Scoring Engine (Server-Side)

```
// server/services/scoring.ts

interface ScoringInput {
  drawnLosses: DrawnLoss[]
  answerKey: LossZone[]
  production: ProductionDay[]
  timeUsedSec: number
  normalCompletionTimeSec: number
}

function calculateScore(input: ScoringInput): GameResults {
  const { drawnLosses, answerKey, production, timeUsedSec, normalCompletionTimeSec } = input
  let totalScore = 0
  let categoryPoints = 0
  let subcategoryPoints = 0
  let sizePoints = 0
  const breakdown = []

  // Match drawn losses to answer key losses
  const matched = matchLossesToAnswerKey(drawnLosses, answerKey)

  for (const match of matched) {
    if (match.type === 'matched') {
      // Player found a real loss
      const { playerLoss, answerLoss } = match

      // Level 1: Category (+10 / -5)
      const catCorrect = playerLoss.category === answerLoss.category
      const catPts = catCorrect ? 10 : -5
      categoryPoints += catPts

      // Level 2: Subcategory (+5 / -2)
      const subCorrect = playerLoss.subcategory === answerLoss.subcategory
```

```
    const subPts = subCorrect ? 5 : -2
    subcategoryPoints += subPts

    // Level 3: Size accuracy (0-10)
    const accuracy = calculateSizeAccuracy(playerLoss.lossSizeKwh, answerLoss.lossSizeKwh)
    const sizePts = Math.round(Math.max(0, (accuracy - 0.80) / 0.20 * 10))
    sizePoints += sizePts

    const total = catPts + subPts + sizePts
    totalScore += total

    breakdown.push({
      type: catCorrect ? 'correct' : 'wrong_category',
      playerLoss, answerKeyLoss: answerLoss,
      categoryPoints: catPts, subcategoryPoints: subPts,
      sizePoints: sizePts, sizeAccuracy: accuracy,
      totalPoints: total
    })
  } else if (match.type === 'false_loss') {
    // Player drew a loss where there isn't one
    totalScore -= 5
    breakdown.push({
      type: 'false_loss',
      playerLoss: match.playerLoss, answerKeyLoss: null,
      categoryPoints: 0, subcategoryPoints: 0,
      sizePoints: 0, sizeAccuracy: 0,
      totalPoints: -5
    })
  } else if (match.type === 'missed') {
    // Answer key loss that player didn't find
    breakdown.push({
      type: 'missed',
      playerLoss: null, answerKeyLoss: match.answerLoss,
      categoryPoints: 0, subcategoryPoints: 0,
      sizePoints: 0, sizeAccuracy: 0,
      totalPoints: 0
    })
  }
}

// Time bonus/penalty (vs. normal completion time)
const timeDiff = normalCompletionTimeSec - timeUsedSec  // positive = faster
let timeBonus
if (timeDiff >= 0) {
  timeBonus = timeDiff  // +1 point per second faster
} else {
  timeBonus = Math.ceil(timeDiff / 5)  // -1 point per 5 seconds slower (negative)
}
totalScore += timeBonus

// Revenue
const correctLosses = breakdown.filter(b => b.type === 'correct' || b.type === 'wrong_category')
const revenueFound = correctLosses.reduce((sum, b) => sum + (b.answerKeyLoss?.revenueLossEur ?? 0), 0)
const totalRevenueLoss = answerKey.reduce((sum, l) => sum + l.revenueLossEur, 0)

return {
```

```
    totalScore, rank: 0, totalPlayers: 0,
    categoryPoints, subcategoryPoints, sizePoints, timeBonus,
    revenueFound, revenueMissed: totalRevenueLoss - revenueFound,
    totalRevenueLoss, breakdown
  }
}
```

Loss Matching Algorithm

A player's drawn loss "matches" an answer key loss if their date ranges **overlap significantly** (>50% overlap with the answer key's range):

```
function matchLossesToAnswerKey(drawn: DrawnLoss[], answerKey: LossZone[]) {
  const results = []
  const matchedAnswerIds = new Set<number>()

  for (const playerLoss of drawn) {
    // Find the best-matching answer key loss
    let bestMatch = null
    let bestOverlap = 0

    for (const answerLoss of answerKey) {
      if (matchedAnswerIds.has(answerLoss.zoneId)) continue
      const overlap = calculateDateOverlap(playerLoss, answerLoss)
      const overlapRatio = overlap / getDateRangeDays(answerLoss)
      if (overlapRatio > 0.5 && overlapRatio > bestOverlap) {
        bestMatch = answerLoss
        bestOverlap = overlapRatio
      }
    }

    if (bestMatch) {
      matchedAnswerIds.add(bestMatch.zoneId)
      results.push({ type: 'matched', playerLoss, answerLoss: bestMatch })
    } else {
      results.push({ type: 'false_loss', playerLoss })
    }
  }

  // Add missed losses
  for (const answerLoss of answerKey) {
    if (!matchedAnswerIds.has(answerLoss.zoneId)) {
      results.push({ type: 'missed', answerLoss })
    }
  }

  return results
}
```

Size Accuracy Calculation

```
function calculateSizeAccuracy(playerSizeKwh: number, actualSizeKwh: number): number {
  if (actualSizeKwh === 0) return playerSizeKwh === 0 ? 1.0 : 0.0
  return Math.max(0, 1 - Math.abs(playerSizeKwh - actualSizeKwh) / actualSizeKwh)
}
```

```
// Points: linear from 80% (0 pts) to 100% (10 pts)
// Formula: max(0, (accuracy - 0.80) / 0.20 * 10), rounded
```

Chart Implementation

uPlot Configuration

The game chart is a customized uPlot instance showing:

1. **Production bars** (primary series) — daily kWh, same visual as Naia aggregated
2. **Rain bars** (inverted, top of chart) — same as Naia slots 1-2
3. **Flow line** (overlaid) — same as Naia slots 3-5
4. **Maintenance dots** — rendered as circle markers on the production series
5. **Additional indicators** — toggled on/off, rendered as additional lines or bars
6. **Drawing overlay** — HTML5 Canvas layer on top of uPlot for the player's drawn lines

Drawing Layer

A transparent canvas element positioned exactly over the uPlot chart area. Handles:

```
// Drawing interaction
class DrawingLayer {
  private canvas: HTMLCanvasElement
  private chart: uPlot
  private currentLoss: { startIdx: number, points: number[] } | null

  onMouseDown(e: MouseEvent) {
    // Convert pixel X to date index
    const dateIdx = this.pixelToDateIndex(e.offsetX)
    this.currentLoss = { startIdx: dateIdx, points: [] }
  }

  onMouseMove(e: MouseEvent) {
    if (!this.currentLoss) return
    // Convert pixel Y to kWh value (this is the expected production level)
    const kwhValue = this.pixelToKwh(e.offsetY)
    // Convert pixel X to date index
    const dateIdx = this.pixelToDateIndex(e.offsetX)
    // Store the expected value for this day
    this.currentLoss.points[dateIdx - this.currentLoss.startIdx] = kwhValue
    this.redraw()
  }

  onMouseUp(e: MouseEvent) {
    if (!this.currentLoss) return
    const endIdx = this.pixelToDateIndex(e.offsetX)
    // Calculate loss area (gap between drawn line and actual production)
    const lossSizeKwh = this.calculateLossArea(this.currentLoss)
    // Emit event to open cause selector
    this.emit('lossDrawn', {
      startDate: this.dates[this.currentLoss.startIdx],
      endDate: this.dates[endIdx],
      expectedKwhPerDay: this.currentLoss.points,
      lossSizeKwh
    })
  }
```

```
  private calculateLossArea(drawing: Drawing): number {
    let total = 0
    for (let i = 0; i < drawing.points.length; i++) {
      const expected = drawing.points[i]
      const actual = this.production[drawing.startIdx + i].energyKwh
      total += Math.max(0, expected - actual)
    }
    return total
  }

  redraw() {
    // Clear canvas
    // Draw all confirmed loss areas (shaded regions)
    // Draw current in-progress loss area
    // Draw maintenance dots
  }
}
```

Simplified Drawing Mode

For players who want to be faster:

1. Click on a start date
2. Click on an end date
3. System draws a flat line at the average of surrounding normal days
4. Player can drag the line up/down to adjust
5. Loss area updates in real-time

Maintenance Dot Rendering

```
// Rendered as circles on the production bars at specific dates
hooks: {
  draw: [
    (u: uPlot) => {
      const ctx = u.ctx
      for (const event of maintenanceEvents) {
        const x = u.valToPos(dateToTimestamp(event.date), 'x', true)
        const y = u.valToPos(getProductionForDate(event.date), 'y', true)

        // Draw dot
        ctx.beginPath()
        ctx.arc(x, y - 8, 6, 0, Math.PI * 2)
        ctx.fillStyle = '#1A5F7A'
        ctx.fill()
        ctx.strokeStyle = '#fff'
        ctx.lineWidth = 2
        ctx.stroke()
      }
    }
  ]
}

// Click detection: check distance from click to each dot
canvas.addEventListener('click', (e) => {
  for (const event of maintenanceEvents) {
    const dotX = u.valToPos(dateToTimestamp(event.date), 'x', true)
```

```
    const dotY = u.valToPos(getProductionForDate(event.date), 'y', true) - 8
    const dist = Math.sqrt((e.offsetX - dotX) ** 2 + (e.offsetY - dotY) ** 2)
    if (dist < 12) {
      showMaintenancePopup(event, dotX, dotY)
      return
    }
  }
})
```

Google Sheets Sync (with Write Verification)

See chapter 05 for the detailed sync + verification logic. Key points:

- After writing a row, the console **reads it back** to verify it was persisted
- Unverified and pending rows are retried every 30 seconds
- When `MULTI_CONSOLE=true`, the Classement reads from Sheets (combined leaderboard from all consoles)
- When offline, falls back to local SQLite leaderboard with a "⚠ Mode local" indicator

```
// server/services/sheets-sync.ts
import { google } from 'googleapis'

const auth = new google.auth.GoogleAuth({
  keyFile: process.env.GOOGLE_CREDENTIALS_PATH,
  scopes: ['https://www.googleapis.com/auth/spreadsheets']
})
const sheets = google.sheets({ version: 'v4', auth })

// Write + verify — see chapter 05 for full implementation
async function syncAndVerify(session: GameSession, participant: Participant) {
  // 1. Append row (includes session.id and CONSOLE_ID for deduplication)
  // 2. Read back to verify session.id appears in the sheet
  // 3. Mark as 'verified', 'unverified', or keep 'pending'
}

// Background retry every 30 seconds
setInterval(() => retryPendingAndUnverified(), 30_000)
```

Deployment (Local)

Build & Run

```
# One-time setup
npm install
npm run build

# Run on game day
node .output/server/index.mjs
# → http://localhost:3000 (game)
# → http://localhost:3000/scoreboard (leaderboard)
```

Windows Convenience Script (`start.bat`)

```
@echo off
echo Starting Naia Conference Game...
cd /d "%~dp0"
start "" "http://localhost:3000"
timeout /t 2 >nul
start "" "http://localhost:3000/scoreboard"
node .output/server/index.mjs
```

Environment Variables

```
# .env
CONFERENCE_ID=hydro-2025-grenoble
ADMIN_SECRET=change-me

# Google Sheets sync (required for multi-console, optional for single)
GOOGLE_SHEETS_ID=1abc...xyz
GOOGLE_CREDENTIALS_PATH=./credentials/service-account.json
CONSOLE_ID=C1                    # C1 or C2 — identifies this console
MULTI_CONSOLE=false              # true = leaderboard reads from Sheets (combined)

# Email via Infomaniak SMTP (game works without)
SMTP_HOST=mail.infomaniak.com
SMTP_PORT=587
SMTP_USER=game@naia.energy
SMTP_PASSWORD=xxxx
SMTP_FROM=game@naia.energy
```

---

07 - Conference Setup

Hardware

Game Station (Single PC Setup)

Item	Spec	Notes
Screen (game)	27" monitor, 1920x1080, IPS	Large enough for spectators. IPS for viewing angles
Screen (scoreboard)	40-55" TV	Connected as extended display to same PC
Mouse	Wired USB	Wireless = battery risk. For drawing + clicking
Computer	Laptop or mini-PC, 8GB+ RAM	Game runs locally — needs Node.js installed
HDMI cable	**5 meters**	Long enough to reach the TV from the arcade cabinet
HDMI adapter	USB-C → HDMI (if needed)	Depends on mini-PC/laptop ports
Power	4 outlets	PC + monitor + TV + LED transformer, use surge-protected strip

No separate computer for scoreboard. Both the game and scoreboard run on the same machine — game on the primary screen, scoreboard in a second browser window on the TV.

No keyboard needed. All text input (email, name) uses an on-screen virtual keyboard operated by mouse. This keeps the arcade cabinet clean and prevents theft/damage.

Internet (Optional)

Situation	What works
Good WiFi	Google Sheets sync + results email work
Spotty WiFi	Game works perfectly, sync catches up when connection returns
No internet	Game works 100%. Export CSV from admin panel at end of day

Internet is **nice-to-have** for a single console, but **required for 2 consoles** (to sync the shared leaderboard via Google Sheets). Each console works 100% offline for gameplay — only the combined Classement needs WiFi. If WiFi drops, each console falls back to its own local leaderboard.

Multi-Console Setup (2 Arcade Cabinets)

When running 2 consoles at the same booth:

1. Both PCs connect to conference WiFi
2. Both run the same Nuxt app with different `CONSOLE_ID` (`C1` and `C2`) in `.env`
3. Both write scores to the same Google Sheet (with write verification)
4. The Classement on each TV shows the **combined** leaderboard from both consoles
5. If WiFi drops: gameplay continues, each console shows its local leaderboard with "⚠ Mode local"
6. When WiFi returns: unsynced scores sync automatically, combined leaderboard rebuilds

Software Setup

Pre-Install on Game PC

```
# 1. Install Node.js (v20+)
# 2. Copy the game folder to the PC
# 3. Build (one-time)
cd naia-conference-game
npm install
npm run build

# 4. Test
node .output/server/index.mjs
# → open http://localhost:3000
```

Game Day Startup

Windows — double-click `start.bat` :

```
@echo off
echo Starting Naia Conference Game...
cd /d "%~dp0"
start "" "http://localhost:3000"
timeout /t 2 >nul
start "" "http://localhost:3000/classement"
node .output/server/index.mjs
```

Manual startup:

```
# Terminal
cd naia-conference-game
```

```
node .output/server/index.mjs

# Then open:
# Primary screen: http://localhost:3000
# TV (drag to second screen): http://localhost:3000/classement → F11 for fullscreen
```

Kiosk Mode (Optional)

```
# Chrome kiosk — game (primary screen)
chrome --kiosk --disable-session-crashed-bubble \
  --disable-infobars --noerrdialogs \
  --app="http://localhost:3000"

# Chrome kiosk — scoreboard (TV)
chrome --kiosk \
  --app="http://localhost:3000/classement?conference=hydro-2025"
```

Staffing

Game Host (1 person, always at booth)

- Invites visitors: "Can you spot why this plant lost EUR 11,000?"
- Explains drawing mechanic quickly: "Tracez où la production aurait dû être"
- Manages queue (switch to Quick Play if >3 waiting)
- Resets game if needed (F5 to refresh)

Sales Person (1 person, during busy times)

- Engages players after results: "You found EUR 8,000 — imagine Naia doing this for YOUR plant"
- Books demos, captures leads
- Handles deeper product questions

Staff Quick Reference

```
NAIA GAME - RÉFÉRENCE RAPIDE
────────────────────────────
Lancer le jeu :   Cliquer « JOUER »
Réinitialiser :   F5 pour rafraîchir le navigateur
Admin :           http://localhost:3000/admin?key=XXX
Export données :  Admin → Export CSV
Forcer sync :     Admin → Forcer sync Google Sheets
Redémarrer :      Fermer terminal, double-clic start.bat
Écran bloqué :    Ctrl+Shift+Esc → redémarrer Chrome
```

Pre-Conference Checklist

1 Week Before

- [ ] All scenarios loaded and tested end-to-end
- [ ] Intro videos for each scenario recorded and tested
- [ ] Game PC prepared: Node.js installed, game built, tested locally
- [ ] Google Sheets sync tested (if using)
- [ ] Email sending verified with test emails (if using)
- [ ] Admin panel accessible to booth staff

- [ ] Hardware packed (cables, adapters, spare mouse)

Day Before

- [ ] Set up booth: monitor, TV, PC
- [ ] Connect TV as extended display, verify resolution
- [ ] Run `start.bat` — verify game on primary screen
- [ ] Drag scoreboard window to TV, press F11
- [ ] Run a full test game on actual hardware
- [ ] Test drawing mechanic with mouse
- [ ] Verify intro video plays for each scenario
- [ ] Check Google Sheets sync (if WiFi available)
- [ ] Print staff reference cards
- [ ] Print QR codes for Naia signup CTA

Morning Of

- [ ] Power on PC, monitor, TV
- [ ] Run `start.bat`
- [ ] Drag scoreboard to TV, F11
- [ ] Run one staff test game → verify full flow (video → draw → score → results)
- [ ] Clear test data from admin panel
- [ ] Confirm admin panel works on staff phones (over WiFi or localhost)

End of Day

- [ ] Export participant CSV from admin
- [ ] Screenshot the final scoreboard
- [ ] Check Google Sheets for completeness
- [ ] Keep everything connected for next day

After Conference

- [ ] Import leads into CRM (within 2 days)
- [ ] Send follow-up email to marketing-consented participants (within 3 days)
- [ ] Share leaderboard on LinkedIn (within 1 day)
- [ ] Internal debrief
- [ ] Delete participant data after 90 days

Budget

Item	Cost	Notes
27" monitor	~EUR 200	Or use existing
TV for scoreboard	~EUR 300	Or rent from venue AV
TV stand	~EUR 50-100	Foldable
HDMI cable 5m + adapters	~EUR 25	Long run to TV
Wired USB mouse (×2)	~EUR 20	One spare
Infomaniak email	Included	Already in naia.energy hosting
Google Sheets	Free	Part of Google Workspace

Total hardware	**~EUR 300-650**	Depending on what's available

Savings vs. previous spec: No separate computer for scoreboard (saves ~EUR 100-200). No cloud hosting costs. No HDMI stick needed.

Metrics

During Conference

- Total games played
- Average score
- Completion rate (finished vs. abandoned)
- Queue length (observed)
- Most common false loss areas (players misidentifying normal production as losses)

Post-Conference

- Email open rate (if emails sent)
- CTA click-through rate
- Demo bookings from game leads
- Trial signups attributed to game
- Social media mentions

Risk Matrix

Risk	Likelihood	Impact	Mitigation
PC failure	Low	High	Backup laptop with game pre-installed
TV failure	Low	Medium	Game works without scoreboard TV
Drawing mechanic confusing	Medium	Medium	Game host explains; simplified drawing mode available
Nobody plays	Low	Medium	Game host actively invites; scoreboard attracts
Long queue	Medium	Low	Switch to Quick Play mode (3 min)
Bug during game	Medium	Low	F5 reset; admin panel for fixes
Data loss	Low	High	SQLite file backed up + Google Sheets sync
Internet unavailable	High	Low	Game is 100% local. Export CSV at end of day

---

08 - Hardware Build: Tabletop Arcade Cabinet

Overview

A custom tabletop arcade cabinet houses the game screen and provides a comfortable, eye-catching setup for conference play. It sits on a standard table (90cm height) and is designed for a standing player of average height (180cm).

The cabinet is built from **poplar plywood 15mm** panels, with a slot for the Acer monitor, a flat mouse shelf, and a Naia-branded illuminated logo panel (lightbox).

> ***Interactive panel drawings***: *Open* `cabinet-panels.html` *in a browser for detailed SVG diagrams of all panels with exact dimensions, assembled front/perspective views, cut list with weights, and transport crate packing layout.*

---

1. Screen Specifications

Acer Monitor

Property	Value
Display area	47.7 cm wide × 30.0 cm high
Bezel — left	1.4 cm
Bezel — right	1.4 cm
Bezel — top	1.4 cm
Bezel — bottom	1.8 cm
Total outer dimensions	**50.5 cm wide × 33.2 cm high**
Panel depth (thickest)	~5.5 cm (at hump on back)
Stand height range	5 cm to 20 cm (from tabletop to bottom of screen)
Inputs	HDMI, VGA, DVI
Power	IEC C14 (standard PC power cable), EU plug

For the cabinet: The monitor **keeps its stand** (non-removable). It sits on an internal shelf inside the cabinet, with the screen visible through a window cutout. The stand's built-in height adjustment (5-20cm) and tilt (~20°) are used to position the screen correctly.

Screen Window Dimensions

The cabinet's front panel has a **window opening** sized to the visible display area only. The monitor bezels sit **behind** the plywood front panel (the panel overlaps the bezels, framing the screen cleanly).

Dimension	Value
Window width	48.7 cm (display 47.7 + 0.5 cm each side)
Window height	31.0 cm (display 30.0 + 0.5 cm each side)

---

2. Ergonomic Calculations

Standing Player — 180cm Height

Parameter	Value	Source
Player height	180 cm	Given (average)
Eye height	~168 cm	~93% of height
Table height	90 cm	Given
Comfortable viewing angle	15-20° below horizontal	Ergonomic standard
Screen center above table	**~65 cm**	Target

---

3. Cabinet Design

Material: Poplar Plywood 15mm

Property	Value

Material	Poplar plywood
Thickness	15mm (1.5cm)
Weight	~6 kg/m²
Finish	Paint Naia Blue (#1A5F7A) + vinyl

Why poplar plywood over MDF? ~50% lighter (6 kg/m² vs. 12 kg/m² for 18mm MDF) at equivalent structural strength. Easier to transport. Screws hold better than in MDF.

Overall Dimensions

Dimension	Value
Cabinet width	58 cm
Inner width (between side panels)	55 cm (58 - 2×1.5)
Height at front	95 cm
Height at back	80 cm
Depth	50 cm
Front panel tilt	20° backward from vertical
Front panel surface length	82 cm
Mouse shelf total width	108 cm (58 + 2×25 overhang)
Back	**Open** (no back panel)

Key Geometry

The front panel is tilted 20° backward — bottom at the front edge, top receding 28cm into the cabinet. Above it, a vertical logo panel faces the player directly. The roof slopes from 95cm (front) to 80cm (back).

```
Side profile points (depth, height in cm):
A (0, 0)      — front bottom
B (28.0, 77.1) — top of front panel (tilted)
C (0, 77.1)    — logo panel bottom / front edge at that height
D (0, 95)      — logo panel top / roof front
E (50, 80)     — roof back / back top
F (50, 0)      — back bottom
```

Assembled Views

See `cabinet-panels.html` for interactive SVG views:

- **Front view**: Shows the cabinet as the player sees it — logo lightbox panel at top, front panel with screen window, mouse shelf with overhangs
- **45° perspective view**: Shows the tilted front panel receding backward, side panel profile, angled roof, and the screen visible through the window cutout

---

4. Panel Cut List

All panels cut from **poplar plywood 15mm**. See `cabinet-panels.html` for detailed drawings with exact dimensions per panel.

Panel A: Side Panels (×2, mirrored)

Complex 6-point shape defining the cabinet profile.

Edge	From → To	Length
Front bottom → Front panel top	A→B	82 cm (along 20° tilt)
Front panel top → Logo bottom	B→C	28.0 cm (horizontal, open — no soffit panel)
Logo bottom → Logo top	C→D	17.9 cm (vertical, front edge)
Logo top → Roof back	D→E	52.2 cm (roof slope)
Back edge	E→F	80 cm (vertical)
Bottom	F→A	50 cm (horizontal)

Features:

- Slot markers at 8, 12, and 16cm height for adjustable mouse shelf position
- Monitor shelf support at Y=38cm
- Area: ~0.36 m^2 each
- Weight: ~2.2 kg each

Panel D: Front Panel (×1)

Dimension	Value
Width	55 cm (inner width)
Height	82 cm (surface length along 20° tilt)
Screen window cutout	48.7 × 31.0 cm
Material	Poplar plywood 15mm
Weight	~2.3 kg

Tilted 20° backward. The bottom edge sits at the front, the top edge recedes 28cm into the cabinet.

Panel E: Roof Panel (×1)

Dimension	Value
Width	55 cm
Length	52.2 cm (slope: $\sqrt{(50^2 + 15^2)}$)
Angle	16.7° from horizontal
Weight	~1.7 kg

Angled panel spanning from front top (95cm) to back top (80cm).

Panel F: Logo Panel — Illuminated Lightbox (×1)

Dimension	Value
Width	55 cm
Height	17.9 cm

Construction	Lightbox (see below)
Weight	~0.6 kg

Vertical panel at the front of the cabinet, above the tilted front panel. This is the main branding surface — the first thing visitors see. **Built as an illuminated lightbox** with backlit Naia logo.

Lightbox Construction (3 layers)

```
┌──────────────────────────────────┐
│  Front: 3mm opal acrylic         │  ← diffuses LED light evenly
│  ────────────────────────────────  │
│  Spacer: 2cm aluminum U-channel  │  ← frame around edges
│  ────────────────────────────────  │
│  Back: 3mm white foamboard + LED │  ← LED strip in zigzag pattern
└──────────────────────────────────┘
```

Component	Spec	Source (Belgium)
Opal acrylic XT 3mm	55 × 17.9 cm, cut to size	Kunststofplaten.be — Plexiglas opaalwit XT 3mm — free custom cutting
White foamboard 3mm	55 × 17.9 cm	Any craft store (Action, Pipoos, Ava) — ~€3
LED strip 12V	120 LEDs/m, 4000K neutral white, ~1.2m needed	Leds-store.be — LED strip 120 LEDs 4000K
Aluminum U-channel 20mm	~1.5m (perimeter frame)	LedprofielKoning.be
24V 60W transformer	Powers the LED strip, plugs into power strip in cabinet	Leds-store.be — transformator 24V 60W
Vinyl decal	Naia hydro logo + wave design, 55×17.9cm	Drukland.be — vinyl sticker op maat — 3 stuks bestellen (1 + reserve)

Assembly: Apply vinyl decal on the *inside* of the opal acrylic (protected, backlit evenly). Mount LED strip on foamboard in zigzag pattern. Assemble with U-channel spacer frame. Connect to 12V supply.

Total lightbox cost: ~€25-40

Panel H: Monitor Shelf (×1)

Dimension	Value
Width	55 cm
Depth	30 cm
Height (mounted at)	38 cm
Weight	~1.0 kg

Structural shelf — supports the monitor on its stand (~5 kg). Must be sturdy.

Panel M: Mouse Shelf (×1)

Dimension	Value

Total width	108 cm (25 + 58 + 25)
Depth	25 cm
Weight	~1.6 kg

Single wide panel that slides through slots in both side panels. The center section passes through the cabinet, the 25cm overhangs on each side provide mouse areas. Three slot height options: 8, 12, or 16cm above table. Mouse pads placed on each overhang.

Cut List Summary

ID	Panel	Qty	Dimensions	Weight	Material
A	Side panel	2	Complex 6-point shape	2×2.2 = 4.4 kg	Poplar ply 15mm
D	Front panel	1	55 × 82 cm (window cutout)	2.3 kg	Poplar ply 15mm
E	Roof panel	1	55 × 52.2 cm	1.7 kg	Poplar ply 15mm
F	Logo panel (lightbox)	1	55 × 17.9 cm	0.6 kg	Acrylic + foamboard + LED
H	Monitor shelf	1	55 × 30 cm	1.0 kg	Poplar ply 15mm
M	Mouse shelf	1	108 × 25 cm	1.6 kg	Poplar ply 15mm
	Total cabinet	**7 panels**		**~11 kg**	

No base panel — the structure is stable enough with the roof panel, monitor shelf, front panel, logo panel, and mouse shelf providing 5 connection points between the side panels.

---

5. Assembly Order

1. **Glue + screw** side panels (A) to roof panel (E) — establishes the cabinet profile
2. **Install** monitor shelf (H) at 38cm height — sturdy, load-bearing
3. **Attach** front panel (D) to the inner face of side panels at the tilted angle
4. **Attach** logo panel lightbox (F) vertically at the front, above the front panel
5. **Insert** mouse shelf (M) through the slots in side panels at chosen height (8/12/16cm)
6. **Finish**: Sand, prime, paint Naia Blue (#1A5F7A), apply vinyl on side panels
7. **Connect** lightbox LED strip to 12V power supply
8. **Place** monitor on shelf (on its stand), adjust stand height + tilt to align with window
9. **Route** cables: HDMI + power through the body, out the open back

No back panel — the open back provides easy access at all times.

---

6. Transport Crate

All panels pack flat in a custom crate for transport. The crate is optimized for minimal volume.

Property	Value
Crate interior	108 × 54.4 × 9.3 cm
Crate exterior	111 × 57.4 × 12.3 cm
Crate material	Pine plywood 9mm (4.5 kg/m^2)
Packing	5 layers, protective textile (3mm felt) between each

Layer Packing

Layer	Contents	Panel thickness
1	Side panel L (95×50)	1.5 cm
2	Side panel R (95×50)	1.5 cm
3	Front panel (82×54.4) + Logo panel (17.9×54.4)	1.5 cm
4	Roof panel (52.2×54.4) + Monitor shelf (54.4×30)	1.5 cm
5	Mouse shelf (108×25)	1.5 cm

Weight Breakdown

Item	Weight
Cabinet panels	~11 kg
Crate (pine plywood 9mm)	~7 kg
Textile padding	~0.5 kg
Total transport weight	**~19 kg**

See `cabinet-panels.html` for detailed crate packing diagrams (top view per layer + side view of stack).

---

7. Hardware Shopping List

Cabinet Build

Item	Cost	Notes
Poplar plywood 15mm sheet (122×244 cm)	~€30-40	One sheet is enough
Carpenter labor (cut + assemble)	~€150-250	Side panels most complex
Paint (Naia Blue spray)	~€15-20	2 cans
Lightbox components (see §4 above)	~€25-40	Acrylic + LED + foamboard + U-channel + vinyl
Vinyl side panels (printed)	~€30-50	Optional branding
Mouse pads (×2)	~€15	For left + right overhangs
Total cabinet build	**~€265-415**	

Electronics & Cables

Item	Cost	Notes
HDMI cable **5 meters**	~€15	Long run from cabinet to scoreboard TV
HDMI adapter (USB-C → HDMI)	~€10	If needed for mini-PC/laptop
USB hub (4-port)	~€10	Mouse + peripherals
Wired USB mouse (×2)	~€20	One spare
Power strip (4-outlet, short cable)	~€10	Inside cabinet

Total electronics	**~€65**	

Transport

Item	Cost	Notes
Pine plywood 9mm for crate	~€15	Cut to crate dimensions
Protective felt/textile	~€10	Between panels
Crate hinges + clasps	~€15	For reusable crate
Total transport	**~€40**	

What You Already Have

- Acer monitor (from photos)
- Wired mouse
- Laptop or mini-PC (for running the game)

What You Might Add

- Mini-PC (Intel NUC or similar): ~€200-400 (if not using a laptop)
- Scoreboard TV (40-55"): ~€300 (or rent from venue)
- TV stand: ~€50-100

---

8. Monitor Placement

The monitor **stays on its stand** — no VESA mount, no brackets. It sits on shelf H inside the cabinet.

Setup Steps

1. Place the monitor (on its stand) on shelf H
2. Center it so the screen aligns with the window cutout in front panel D
3. Adjust the stand height to ~10 cm (screen bottom at window bottom edge)
4. Tilt the screen back ~20° using the stand's tilt mechanism
5. Plug in HDMI + power cables, route through cabinet interior, out the open back

---

9. Branding & Finish

Color Scheme

- **Cabinet body**: Naia Blue (#1A5F7A) — spray painted or vinyl wrapped
- **Logo panel**: Illuminated lightbox with Naia hydro logo (blue/purple wave design on opal acrylic, backlit with LED)
- **Side panels**: Printed vinyl with Naia branding / hydro imagery
- **Mouse shelves**: Black or dark gray accent

---

10. Scoreboard TV

The scoreboard TV (40-55") is separate from the arcade cabinet.

Option	Notes
TV on a stand behind/beside the cabinet	Simple, portable
Wall-mounted (if venue allows)	Visible from distance

Connected to the same mini-PC via **5m HDMI cable** (or second HDMI output), showing `localhost:3000/classement` in Chrome fullscreen.